Intra-Question Label Leakage in Knowledge Tracing with Multi-KC Questions: A Controlled Study on XES3G5M

Nhat-Anh Nguyen-Bui[1,4], Anh-Kiet Vo[1,4], Thu Nguyen[1,4] (✉), Khoa Tan Vo[1,4], Phuc Nguyen[2,4], and Tu-Anh Nguyen-Hoang[3,4]

[1] Faculty of Information Science and Engineering, University of Information Technology, Ho Chi Minh City, Vietnam
[2] Faculty of Information Systems, University of Economics and Law, Ho Chi Minh City, Vietnam
[3] Information Security Laboratory, University of Information Technology, Ho Chi Minh City, Vietnam
[4] Vietnam National University, Ho Chi Minh City, Vietnam
{23520056,23520824}@gm.uit.edu.vn, {thunta,khoavt}@uit.edu.vn, phucnq@uel.edu.vn, anhnht@uit.edu.vn

Abstract. Knowledge tracing (KT) predicts whether a student will answer the next question correctly. Many KT models expand a question with several knowledge components (KCs) into one training token per KC, each carrying the question's label. The pyKT benchmark trains this way but evaluates every KC from the history before the question. This mismatch lets the model read the label it must predict and learn to copy it from the second KC on. We propose Pre-Question Conditioning (PQC), a training rule rather than an architecture change: a question's label must not enter the state used to predict it, and all its KCs are predicted from the state before the question. On XES3G5M, standard DKT has an AUC 0.098 lower on the second and later KCs of multi-KC questions than on the first. PQC raises question-level AUC from 0.7841 to 0.7891 (+0.0050, all 15 runs) and lowers ECE from 0.0111 to 0.0047. Removing the label from the context gives +0.0045, reading the pre-question state a further +0.0011. The gain is concentrated in multi-KC questions (+0.032 AUC versus +0.001 on single-KC questions). Controls on DKT (shuffled labels, intervention test) support copying and show that PQC variants do not leak. On simpleKT, the same rule raises AUC from 0.8121 to 0.8164 (+0.0043, 5 of 5 folds) and removes the gap between first and later KCs, with indirect evidence for the mechanism. It lowers AUC on single-KC questions (−0.0027), mostly on immediate repeats, which our segment convention hides from the model.

Keywords: Knowledge Tracing · Label leakage · Multi-KC questions · Evaluation Protocol · Training Rule

1 Introduction

Knowledge tracing (KT) predicts the probability that a student answers the next question correctly from their learning history. Most KT models work at the level of knowledge components (KCs), the skills that a question exercises, and a question can cover more than one of them: in XES3G5M [1], a large K-12 mathematics dataset with 5,549,635 interactions from about 18,066 students on 7,652 questions over 865 leaf-node KCs, each question is associated with 1.164 KCs on average.

The pyKT benchmark [2] treats such questions asymmetrically. In training, a multi-KC question is expanded into one token per KC, and every token carries the label of the question. In evaluation (all-in-one), every KC is predicted from the history before the question, and the KC predictions are then aggregated into a single question-level probability. Hence, when the second KC of a question is predicted in training, the first KC and the label to be predicted have already been read. A model can learn to copy the label of the preceding token instead of deriving it from the student's past performance, a shortcut that is useless at evaluation, where the label is never visible.

Xiong et al. [3] showed that DKT can exploit the repeated rows of multi-skill questions, and Badran and Preisach [4,5] identify label leakage in KC expansion and propose remedies (masked labels, fused updates, a question mask for attention). Still missing are a controlled measurement under the pyKT all-in-one protocol and a separation of what a remedy must change.

The remedy is a rule on the information available to each KC prediction, called Pre-Question Conditioning (PQC): (i) during training, the context of a prediction contains no information about the label of its own question; (ii) all KCs of a question are predicted from the same context as at evaluation, the history before the question. Condition (ii) implies (i), but some implementations meet (i) without (ii), so each step can be measured separately. PQC constrains what the model reads, not the architecture. It is instantiated on DKT [6] with five variants, and on simpleKT [7] with a question mask on the attention.

On XES3G5M, standard DKT (3 seeds, 5 folds) is 0.098 AUC lower on the second and later KCs of multi-KC questions than on the first (0.7029 against 0.8008). PQC removes most of this gap and raises question-level AUC from 0.7841 to 0.7891 (+0.0050, 15 of 15 runs); on simpleKT (5 folds) it raises AUC by +0.0043 (5 of 5 runs). In both models the gain is concentrated in multi-KC questions (+0.032 and +0.024 AUC). On simpleKT the rule costs 0.0027 AUC on single-KC questions, concentrated in immediate repeats of the same question, which our segment convention hides from the model (Sect. 6.6).

Our contributions are:

1. We measure intra-question label leakage under the pyKT all-in-one protocol: about 0.10 AUC is lost on the later KCs of multi-KC questions for DKT and about 0.13 for simpleKT.
2. We state PQC, a two-condition rule that removes it without changing the architecture (+0.0050 AUC on DKT, +0.0043 on simpleKT, with better

Brier score and lower ECE), and decompose its effect: (i) gives +0.0045, (ii) a further +0.0011, and the implementation is a second-order effect (at most 0.0006).
3. We analyse the gain by question group, KC position and position in the student's sequence.

The masked-label, fused-update and question-mask variants come from earlier work [4,5]. Our results concern one dataset and two backbones: DKT carries the main analysis (3 seeds, 5 folds), simpleKT has one seed (5 folds).

2 Related Work

KT models DKT [6] reads the sequence of (KC, label) pairs with an LSTM. DKVMN [8] adds a key–value memory, SAKT [9] replaces the recurrence by attention, AKT [10] adds monotonic attention with a Rasch-style question difficulty, and simpleKT [7] uses standard attention with a question difficulty vector. Question-level models such as QIKT [11] use one step per question, so they have no intra-question label leakage by construction, but they change the architecture and the input. Our rule targets KC-level models, which remain widely used and can be corrected without such a change; in pyKT's expanded sequences, recurrent and attention mechanisms read the earlier KC tokens of the same question, and we measure this only for DKT and simpleKT.

Multi-KC questions, label leakage and protocols. Xiong et al. [3] observed that DKT benefits from repeated rows of multi-skill questions (average AUC 0.81 against 0.74 with one row per question on ASSISTments 2009–2010). Badran and Preisach [4] name label leakage in KC expansion and propose a masked label for the KCs before the last one (DKT-ML), a fused update with one step per question (DKT-Fuse) and a question mask in attention (AKT-QM), the journal extension [5] adds further variants (DKT-ML+, SAKT-ML, AKT-Fuse). They also propose an autoregressive variant (DKT-AD) and report that leakage grows with the number of KCs per question; in their KTbench evaluation the leaking baselines are tested all-in-one and the leak-free variants one-by-one. pyKT [2] standardizes KT evaluation (expanded training, all-in-one late fusion at test time), which makes numbers comparable but leaves training and evaluation different in what the model can read. We keep the pyKT protocol on XES3G5M [1], which provides pyKT baselines.

Position of this work. The variants of Badran and Preisach [5] are taken here as design points, not claimed as our own. This work differs in three ways. First, the remedy is split into two conditions, and their effects are measured separately. Second, every model, leak-free variants included, is evaluated with the same pyKT all-in-one protocol on XES3G5M, with 3 seeds, 5 folds and a bootstrap over students. Third, the remedy is stated as a rule on the context of each prediction, which applies to both a recurrent and an attention backbone while keeping the per-KC loss. We do not propose a new architecture or aim to outperform AKT, simpleKT or QIKT.

3 Problem Formulation

3.1 Notation and the pyKT Protocol

A student's history is a sequence of question attempts $\mathcal{S} = \big((q_t, C_t, r_t)\big)_{t=1}^{T}$, where $q_t \in \mathcal{Q}$ is the question, $C_t \subseteq \mathcal{C}$ its set of KCs, $m_t = |C_t|$, and $r_t \in \{0, 1\}$ is the response. We write $C_t = \{c_{t,1}, \ldots, c_{t,m_t}\}$ in a fixed order. A question with $m_t > 1$ is a multi-KC question, and r_t is a single label for the whole question, not one per KC. The model predicts $P\big(r_{T+1} = 1 \mid q_{T+1}, C_{T+1}, \mathcal{S}\big)$. We follow pyKT [2], which trains at the KC level and evaluates at the question level.

Training. Each q_t is expanded into m_t tokens $(q_t, c_{t,j}, \rho_{t,j})$ whose labels are inherited from the question, $\rho_{t,1} = \cdots = \rho_{t,m_t} = r_t$. A model reads the full token or only $(c_{t,j}, \rho_{t,j})$ (DKT ignores q_t); we write $\tau_{t,j}$ for the part it reads. With $\oplus$ the concatenation of sequences, the history before q_t as read by the model is $\tilde{\mathcal{S}}_{<t} = (\tau_{s,i})_{s<t,\, i\le m_s}$, and the context when the j-th KC of q_t is predicted is

$$\mathcal{X}_{t,j} = \tilde{\mathcal{S}}_{<t} \oplus \big(\tau_{t,i}\big)_{i<j}. \tag{1}$$

Evaluation. Labels exist only per question. Each KC of q_t is predicted independently from $\tilde{\mathcal{S}}_{<t}$ alone, and the KC probabilities are averaged into one question-level prediction in this work (mean late fusion).

3.2 Intra-Question Label Leakage

Training and evaluation do not condition on the same information. In training, the j-th KC of q_t with $j > 1$ is predicted after $(q_t, c_{t,1}, r_t), \ldots, (q_t, c_{t,j-1}, r_t)$ have been read, so the input already contains the label to be predicted, only the first KC of each question is free of this leakage. At evaluation r_t is never visible. The model may therefore learn a shortcut: copy the label of the preceding token whenever it infers that two KCs belong to the same question, instead of deriving r_t from the student's past performance.

4 Training Rule: Pre-Question Conditioning

Training should limit what each KC prediction reads, since expanded training lets a model read r_t before predicting later KCs of q_t while evaluation never does. We call the resulting rule Pre-Question Conditioning (PQC); it constrains the input of the model and is not a new architecture.

4.1 The Rule

Evaluation uses the context $\mathcal{X}^{\text{eval}}_{t,j} = \tilde{\mathcal{S}}_{<t}$. PQC requires, for every q_t and $j \le m_t$:

(i) during training, $\mathcal{X}_{t,j}$ contains no information about r_t;

(ii) all KCs of q_t are predicted from the same context as at evaluation, the history before q_t.

Condition (ii) implies (i); they are stated separately because some implementations meet (i) but not (ii) (DKT-ML below). Training uses the per-KC cross-entropy,

$$\mathcal{L} = -\sum_{t}\sum_{j=1}^{m_t}\big[r_t \log \hat{p}_{t,j} + (1-r_t)\log(1-\hat{p}_{t,j})\big], \quad \hat{p}_{t,j} = P(r_t = 1 \mid q_t, c_{t,j}, \mathcal{X}_{t,j}). \tag{2}$$

With one KC per question the rule changes nothing: the only context is already the history before q_t.

4.2 Instantiation on DKT

DKT [6] embeds tokens $(c_{t,j}, \rho_{t,j})$ as $\mathbf{x}_{t,j}$ and runs an LSTM, $\mathbf{h}_{t,j} = \mathrm{LSTM}(\mathbf{x}_{t,j}, \mathbf{h}_{t,j-1})$, where $\mathbf{h}_{t-1} = \mathbf{h}_{t-1,m_{t-1}} = \mathbf{h}_{t,0}$ is the state left by the previous question. The output is $\mathbf{y}_{t,j} = \sigma(\mathbf{W}\mathbf{h}_{t,j} + \mathbf{b}) \in [0,1]^{|\mathcal{C}|}$.

DKT (baseline). Every token carries r_t and $\hat{p}_{t,j} = \mathbf{y}_{t,j-1}[c_{t,j}]$. For $j > 1$ the context contains r_t, so (i) and (ii) fail.

DKT, leak-free selection. This control is trained exactly as DKT, so (i) and (ii) still fail during training. It differs only in how the checkpoint is chosen: the epoch is selected with the validation criterion of DKT-PQC, which reads no label of the question being predicted, instead of the standard criterion of DKT. It is a control, not a remedy: it tests whether the gain of PQC comes from the rule or only from checkpoint choice.

DKT-ML. A third label MASK replaces $\rho_{t,j}$ for all $j < m_t$ (only the last token keeps r_t), and targets stay r_t. (i) holds, but $c_{t,j}$ is still predicted after earlier KCs of q_t, so (ii) fails. Our MASK embedding adds $|\mathcal{C}|d$ parameters (Badran and Preisach use fewer).

DKT-PQC (ours). Tokens keep the true label and are read sequentially, but every KC of q_t is predicted from the state before the question,

$$\hat{p}_{t,j} = \sigma(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{b})[c_{t,j}], \qquad \mathcal{X}_{t,j} = \tilde{\mathcal{S}}_{<t}. \tag{3}$$

So (ii), hence (i), holds. The tokens of q_t only update the state for q_{t+1}.

DKT-ML-PQC. This variant uses MASK labels as in DKT-ML, with every KC predicted from $\mathbf{h}_{t-1}$. It differs from DKT-PQC only in what the state stores: r_t on every KC token, or only on the last.

DKT-Fuse. This variant feeds the LSTM one token per question, namely the mean of its KC encodings, so that each question takes a single LSTM step: $\bar{\mathbf{x}}_t = \frac{1}{m_t}\sum_j \mathbf{x}_{t,j}$, $\mathbf{h}_t = \mathrm{LSTM}(\bar{\mathbf{x}}_t, \mathbf{h}_{t-1})$, $\hat{p}_{t,j} = \sigma(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{b})[c_{t,j}]$. Both (i) and (ii) hold by construction. In contrast to Badran and Preisach, who average KC predictions before the loss, we keep one loss term per KC and average only at evaluation.

4.3 Instantiation on simpleKT

simpleKT [7] reads the expanded KC-level sequence. For each interaction: $(q_t, c_{t,j}, \rho_{t,j})$, the query/key and value representations are computed as: $\mathbf{x}_{t,j} = \mathbf{z}_{c_{t,j}} + \mathbf{m}_{q_t} \odot \mathbf{v}_{c_{t,j}}$ and $\mathbf{y}_{t,j} = \mathbf{z}_{c_{t,j}} + \mathbf{g}_{\rho_{t,j}}$, respectively. Self-attention is applied over previous interactions, followed by a two-layer fully connected network and a sigmoid output layer to predict the probability of a correct response. In the original model, the j-th KC of question q_t can attend to earlier KCs from the same question. Since these interactions already encode the response r_t and share the same question embedding $\mathbf{m}_{q_t}$, the model can exploit them as a shortcut to infer the target label, violating assumptions (i) and (ii). **simpleKT-PQC** keeps the architecture and modifies only the readable positions, following AKT-QM; interactions belonging to q_t are never read:

$$\mathbf{h}_{t,j} = \mathrm{SelfAttention}\big(Q = \mathbf{x}_{t,j},\ K = (\mathbf{x}_{s,i})_{s<t},\ V = (\mathbf{y}_{s,i})_{s<t}\big). \tag{4}$$

Unlike the DKT-based baselines, which follow pyKT by preserving the original interaction sequence and using an `is_repeat` flag, simpleKT-PQC groups consecutive interactions with the same question ID into a single segment. Consequently, when all readable positions within a segment are masked, the interaction receives a zero-vector representation, as for the first row in the original simpleKT; this happens only when a student's very first interactions are repeated attempts on the same question. The merge has a side effect: when the second of two consecutive attempts at the same question is predicted, the first is hidden although it is legitimate history at test time. This concerns immediate repeats (10 424 of the 1 099 197 graded questions, 0.95%) and is the likely source of the loss in Sect. 6.6. So (ii), hence (i), holds. In Table 1, row i is the target interaction (t, j), row k a past interaction that row i may observe, and $g_i = t$.

4.4 Design Space and Provenance

Table 1 lists all variants. AKT-QM, DKT-ML and DKT-Fuse come from Badran and Preisach [4,5], not from us. We add (a) the split of the rule into (i) and (ii), each measured separately, (b) DKT-PQC, a sequential implementation with one step per KC, and DKT-ML-PQC as an intermediate step, and (c) a controlled all-in-one evaluation on DKT and simpleKT, including the leak-free selection control. DKT-PQC also differs from the autoregressive DKT-AD of Badran and Preisach [4], which replaces earlier true labels by the model's own predictions and still predicts KC j after them: it meets (i) but not (ii).

Table 1. Design space of the training rule. For DKT, $h_{t,j-1}$ is the state after the first $j-1$ KCs of q_t and $h_{t,0}$ the state before q_t. For simpleKT, i is the row being predicted, k a row it may read, g_i the question of row i. (i), (ii): Sect. 4.1. Parameters are for our configuration; the pyKT DKT (841,465) is a separate baseline in Table 2.

Variant	Update by the KCs of q_t	KC j read from	(i)	(ii)	Params	Origin
DKT	one step per KC, true label	$h_{t,j-1}$	no	no	588,465	[6], our impl.
DKT, leak-free selection	as DKT; checkpoint chosen with the validation criterion of DKT-PQC	$h_{t,j-1}$	no	no	588,465	control (ours)
DKT-ML	one step per KC, MASK except last	$h_{t,j-1}$	yes	no	674,965	[4,5]
DKT-ML-PQC	as DKT-ML	$h_{t,0}$	yes	yes	674,965	ours
DKT-PQC	one step per KC, true label	$h_{t,0}$	yes	yes	588,465	ours
DKT-Fuse	one step per question	$h_{t,0}$	yes	yes	588,465	[4,5]; per-KC loss ours
simpleKT	attention over rows $k < i$	rows $k < i$	no	no	3,754,497	[7,2]
simpleKT-PQC	attention over rows with $g_k < g_i$	rows k with $g_k < g_i$	yes	yes	3,754,497	AKT-QM principle [4,5]

5 Experimental Setup

5.1 Dataset, Models and Metrics

We use XES3G5M [1], collected from an online mathematics platform for K-12 students in China: 5 549 635 interactions from 18 066 students on 7 652 questions, annotated with 865 leaf-node KCs. A question has on average 1.1640 KCs, so multi-KC questions are a minority but not negligible: 12.5% of the graded questions (those scored in the metrics, 137 481 of 1 099 197) have more than one KC, and these are the cases where intra-question leakage can occur. We evaluate on a fixed test set of 3 613 students.

The five DKT variants of Table 1 share the same hyperparameters and are trained with 3 seeds (42–44) $\times$ 5 folds, i.e. 15 runs per variant. We also run the pyKT default configuration with 5 folds and seed 42 (5 runs per model) for DKT, simpleKT, simpleKT-PQC, AKT and QIKT; simpleKT-PQC differs from simpleKT only by the mask of Eq. (4), and QIKT (one step per question) is a question-level reference. No hyperparameter was tuned. We report mean and sd over runs.

All models are evaluated with the all-in-one protocol of Section 3.1, and all metrics are computed at the question level, with one probability $\hat{p}_t$ (mean late fusion) and one label $y_t = r_t$ per question: **AUC** (main metric); **Brier** $\frac{1}{N}\sum_t(\hat{p}_t - y_t)^2$; **ECE** with 10 equal-width bins; and **AUPRC-wrong**, the area under the precision–recall curve with the incorrect response as the positive class and $1-\hat{p}_t$ as score. Higher is better for AUC and AUPRC-wrong, lower for Brier and ECE.

Implementation. We use pyKT (commit `eec087e`, with a question-mask option for simpleKT-PQC and a device fix for QIKT) on one NVIDIA T4 GPU (Kag-

gle); the checkpoint is the epoch with the best KC-level validation AUC of pyKT, except for the leak-free selection control. All models are trained with Adam and early stopping on validation AUC (patience 10); other hyperparameters are the pyKT defaults. The DKT variants use pyKT's `is_repeat` flag and do not merge repeated attempts; simpleKT-PQC does (Sect. 4.3). The pre-registered check counted merged positions over all positions of the test file, graded or not: 1.04%, above the 1% threshold set before the runs, which is recorded as a deviation (0.79% on the training file). The 0.95% of Sect. 4.3 is the same phenomenon counted on graded questions only.

5.2 Statistical Testing

We use a paired bootstrap that resamples students, because the questions of one student are correlated. For two runs a and b on the same fold and seed, we draw 1 000 replicates and weight every question by the resampling weight of its student. In each replicate we compute $\Delta = \text{metric}(b) - \text{metric}(a)$ for AUC, Brier score and ECE. We report Δ on the full test set together with the 95% percentile interval (CI) over the replicates. Each comparison yields 15 differences (5 for the pyKT runs), summarized by mean, minimum, maximum and the number of runs whose CI excludes 0. A difference is *clear* if it has the same sign in all runs and its CI excludes 0 in at least 8 of 15 runs (3 of 5), a *trend* if it has the same sign but the CI excludes 0 in fewer runs and *not distinguishable* if the sign changes between runs. Only clear differences are set in bold. The comparisons follow Section 4.4: the steps DKT $\rightarrow$ DKT-ML $\rightarrow$ DKT-ML-PQC $\rightarrow$ DKT-PQC (evaluated on the same questions, so their Δ values add up to DKT-PQC $-$ DKT), the implementations of PQC against DKT and each other, and simpleKT-PQC against simpleKT.

Mechanism analyses. To check the mechanism, we repeat the paired bootstrap on subsets of the test questions defined by two labels. The target label says whether the predicted question has a single KC or multiple KCs. The previous label says whether the question immediately before it has a single KC or multiple KCs, or is unknown; it is unknown when the predicted question is the first graded question of its student (3 613 questions). We cut the test set by the target label alone (2 subsets), by the previous label alone (3 subsets), and by both crossed ($2 \times 3 = 6$ subsets), giving 11 subsets in total. Table 4 reports 9 of them; the two previous-only cuts for a single-KC or a multi-KC previous question are unions of crossed subsets and are omitted. If leakage is the cause, the gain should concentrate where the predicted question has several KCs.

As a second diagnostic, we report the KC-level AUC separately for the first KC and for the second and later KCs of multi-KC questions. Under standard training only the later KCs can see r_t, so a leaking model should be worse on them. It is never used to rank models.

Table 2. Main results on XES3G5M (mean over runs; ± is the sd over runs; DKT variants 15 runs, other models 5 runs). ACC uses a threshold of 0.5 and is not tested. Paired tests: Table 3 (DKT) and Sect. 6.6 (simpleKT). "Ref. AUC": value published for the pyKT model on XES3G5M.

Model	Runs	AUC	ACC	Brier	ECE	Params	Ref. AUC
DKT	15	0.7841 ± 0.0002	0.8172	0.1319	0.0111 ± 0.0015	588,465	–
DKT, leak-free selection	15	0.7840 ± 0.0002	0.8172	0.1319	0.0111 ± 0.0015	588,465	–
DKT-ML	15	0.7886 ± 0.0002	0.8189	0.1305	0.0056 ± 0.0022	674,965	–
DKT-ML-PQC	15	0.7897 ± 0.0002	0.8192	0.1303	0.0047 ± 0.0014	674,965	–
DKT-PQC	15	0.7891 ± 0.0002	0.8191	0.1304	0.0047 ± 0.0018	588,465	–
DKT-Fuse	15	0.7894 ± 0.0001	0.8193	0.1303	0.0041 ± 0.0014	588,465	–
DKT (pyKT)	5	0.7834 ± 0.0001	0.8165	0.1322	0.0120 ± 0.0045	841,465	0.7852
AKT (pyKT)	5	0.8162 ± 0.0008	0.8261	0.1243	0.0172 ± 0.0014	6,824,518	0.8207
QIKT (pyKT)	5	0.8206 ± 0.0001	0.8294	0.1222	0.0076 ± 0.0074	13,290,684	0.8222
simpleKT (pyKT)	5	0.8121 ± 0.0004	0.8235	0.1258	0.0191 ± 0.0017	3,754,497	0.8163
simpleKT-PQC	5	0.8164 ± 0.0004	0.8269	0.1238	0.0081 ± 0.0024	3,754,497	–

5.3 Sanity Checks for Leakage

Five checks verify that label copying is the cause and that the PQC variants do not leak (fold 0, seed 42). The intervention test is run on DKT and DKT-PQC, the other checks on the five DKT variants. *Global shuffle* permutes the training labels over the whole set (validation AUC should be about 0.5). *Shuffle by question* permutes question-level labels and copies each to all KCs of its question: the history carries no signal, but a leaking model can exceed 0.5 by copying the preceding KC. We report the best KC-level validation AUC and the test Brier score (random-label models output almost constant probabilities). *Overfitting*: each variant must fit 4 training sequences, so low scores under shuffling do not reflect an inability to learn. *Intervention test*: in a trained model we flip the label of one random multi-KC question with a next question, in each of 32 training sequences (5 draws), and record the largest change of prediction on the KCs of that question (`inside`) and on the next question (`after`); without leakage `inside` $\leq 10^{-6}$, and `after` > 0 shows the history is still used. *Positive control*: standard DKT must be flagged as leaking and DKT-PQC must pass.

6 Results

DKT numbers are means over 15 runs (3 seeds × 5 folds); numbers for simpleKT, AKT, QIKT and the pyKT DKT are means over 5 runs (5 folds, seed 42). Differences are "clear", "trend" or "not distinguishable" as in Section 5.2; only clear differences are in bold in Tables 3 and 4.

6.1 Main Results and Calibration

Table 2 compares the models. The three PQC implementations of DKT reach an AUC of 0.7891–0.7897 against 0.7841 for standard DKT, and DKT-ML, which only removes the label from the context, lies in between (0.7886). The ordering

Table 3. Paired bootstrap differences $b - a$ on DKT (AUC: mean [min; max] over 15 runs; CI: runs with CI above 0 / below 0; Brier and ECE: mean). Bold: clear difference.

Comparison ($b - a$)	ΔAUC [min; max]	CI	ΔBrier	ΔECE
DKT-ML − DKT (i)	**+0.00448** [+0.00410; +0.00506]	15 / 0	**−0.00136**	**−0.00544**
DKT-ML-PQC − DKT-ML (ii)	**+0.00109** [+0.00046; +0.00148]	15 / 0	**−0.00029**	−0.00092
DKT-PQC − DKT-ML-PQC (label)	**−0.00058** [−0.00121; −0.00005]	0 / 10	+0.00014	−0.00005
DKT-PQC − DKT (total)	**+0.00499** [+0.00473; +0.00543]	15 / 0	**−0.00151**	**−0.00641**
DKT-ML-PQC − DKT	**+0.00557** [+0.00510; +0.00606]	15 / 0	**−0.00165**	**−0.00636**
DKT-Fuse − DKT	**+0.00533** [+0.00483; +0.00568]	15 / 0	**−0.00161**	**−0.00696**
DKT-Fuse − DKT-PQC	**+0.00034** [+0.00010; +0.00061]	8 / 0	**−0.00010**	−0.00055
DKT-Fuse − DKT-ML-PQC	−0.00024 [−0.00059; +0.00026]	0 / 6	+0.00004	−0.00060
DKT-PQC − DKT, leak-free selection	**+0.00506** [+0.00465; +0.00560]	15 / 0	**−0.00153**	**−0.00647**
DKT, leak-free selection − DKT	−0.00007 [−0.00056; +0.00029]	1 / 3	+0.00002	+0.00006

is the same for all three seeds, and the standard deviation over 15 runs (0.0001–0.0002) is about 25 times smaller than the gap between DKT-PQC and DKT. DKT-PQC and DKT-Fuse add no parameters, so the gain does not come from model size; DKT-ML and DKT-ML-PQC add 86 500 (+14.7%) and are not better. On simpleKT, the mask raises AUC by 0.0043 (0.0039–0.0050) and ACC from 0.8235 to 0.8269; AUC, Brier and ECE are clear in 5 of 5 runs.

PQC is not meant to close the gap between backbones: our best DKT variant is 0.022–0.027 below the pyKT simpleKT and AKT, while simpleKT-PQC is at the level of the pyKT AKT (0.8164 against 0.8162; no paired test) and 0.0042 below QIKT. Our pyKT runs are 0.0016–0.0045 below the published values (Table 2), so comparison with them is only indicative.

Calibration. The gain is not limited to ranking: ECE falls from 0.0111 to 0.0041–0.0047 and Brier from 0.1319 to 0.1303–0.1304. The DKT variants with PQC are better calibrated than simpleKT (ECE 0.0191) and AKT (0.0172), although these have a higher AUC. On simpleKT the mask lowers ECE to 0.0081, close to QIKT (0.0076).

6.2 Decomposition and Implementations

Table 3 separates the effect of the rule into the steps of Section 4.1. Removing the label while keeping the sequential reading (condition (i), DKT → DKT-ML) adds 0.0045 AUC, and reading the state before the question (condition (ii), DKT-ML → DKT-ML-PQC) a further 0.0011; both are clear, with the interval excluding 0 in 15 of 15 runs. Replacing MASK by the true label (DKT-ML-PQC → DKT-PQC) lowers AUC by 0.0006 (clear in sign, but small). The three steps add up exactly to the total +0.0050 in AUC, and likewise in Brier and ECE; most of the Brier and ECE gain comes from the first step. The three implementations of the rule differ by at most 0.0006 AUC, about nine times less than the effect of the rule itself (DKT-Fuse is above DKT-PQC by 0.0003, interval excluding 0 in 8 of 15 runs; DKT-Fuse and DKT-ML-PQC cannot be distinguished). We therefore do not rank them: what matters is that (i) and (ii) hold. Finally, selecting the checkpoint of standard DKT with a leak-free

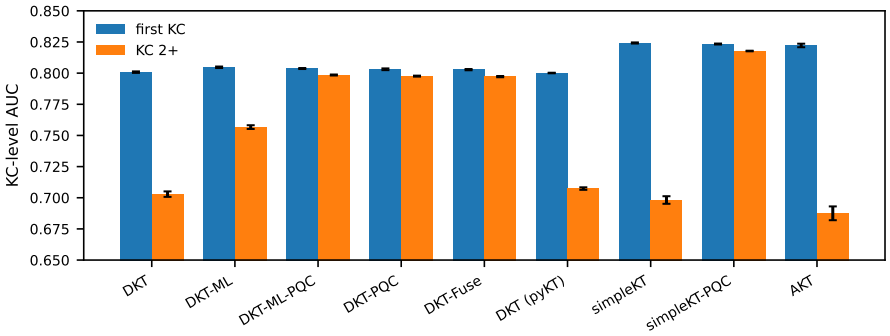


Fig. 1. KC-level AUC in multi-KC questions, first KC against second and later KCs (mean over runs; error bars: sd over runs; DKT variants 15 runs, other models 5 runs). Diagnostic only, not used to rank models.

criterion changes AUC by −0.00007 and the sign changes between runs (not distinguishable); against this variant DKT-PQC is still better by +0.0051 AUC, so the gain does not come from the way the checkpoint is chosen.

6.3 Mechanism: Where the Gain Comes From

AUC by KC position. Figure 1 gives the KC-level AUC in multi-KC questions, for the first KC and for the second and later KCs. Standard DKT is almost 0.10 AUC worse on the later KCs (0.7029) than on the first (0.8008; gap -0.0978 ± 0.0024), and simpleKT and AKT show larger gaps (−0.126, −0.135). This is what label copying predicts: under all-in-one evaluation the previous KC of the same question is not available, so the shortcut is useless exactly where it was learned. The pyKT DKT shows the same drop (−0.093), so it does not come from our implementation. DKT-ML, which still predicts KC j after the earlier KCs of the question, is in between (−0.048). With (ii) the gap is only about −0.005 (DKT variants) and −0.0056 (simpleKT-PQC), which we read as the baseline difficulty of later KCs.

AUC by question group. If leakage is the cause, the gain should be concentrated in multi-KC questions, and Table 4 shows that it is. For DKT-PQC the AUC gain is +0.0321 on multi-KC targets (137 481 questions) against +0.0010 on single-KC targets (961 716); DKT-Fuse and DKT-ML-PQC behave the same way (+0.0320 and +0.0328 against +0.0014 and +0.0016). For a fixed type of target the gain hardly depends on the previous question (+0.0325 and +0.0314 for multi-KC targets), so the target determines the gain, consistent with the leakage explanation. On simpleKT the gain on multi-KC targets is similar (+0.0242, 5 of 5 folds), but on single-KC targets the mask is slightly worse (−0.0027, 5 of 5 folds); Sect. 6.6 shows that this loss is concentrated in immediate repeats.

Table 4. Change in AUC by question group (target / previous question): DKT-PQC − DKT (mean over 15 runs) and simpleKT-PQC − simpleKT (mean over 5 runs). In parentheses: runs with a positive difference; then runs with CI above 0 / below 0. The last block (immediate repeats: the question ID equals that of the previous graded question) is given for simpleKT only, since DKT does not merge repeats (Sect. 4.3). Bold: clear difference.

Target / previous question	#questions	DKT-PQC − DKT (of 15)	simpleKT-PQC − simpleKT (of 5)
1 KC / all	961 716	**+0.0010 (15; 15 / 0)**	**−0.0027 (0; 0 / 5)**
multi-KC / all	137 481	**+0.0321 (15; 15 / 0)**	**+0.0242 (5; 5 / 0)**
1 KC / 1 KC	865 139	**+0.0011 (15; 15 / 0)**	**−0.0031 (0; 0 / 5)**
1 KC / multi-KC	93 251	+0.0006 (14; 2 / 0)	+0.0010 (5; 2 / 0)
multi-KC / 1 KC	93 786	**+0.0325 (15; 15 / 0)**	**+0.0245 (5; 5 / 0)**
multi-KC / multi-KC	43 408	**+0.0314 (15; 15 / 0)**	**+0.0223 (5; 5 / 0)**
1 KC / 1st graded	3 326	**−0.0067 (0; 0 / 9)**	−0.0012 (1; 0 / 0)
multi-KC / 1st graded	287	**+0.0452 (15; 12 / 0)**	**+0.0399 (5; 3 / 0)**
all 1st graded	3 613	+0.0001 (8; 0 / 0)	**+0.0087 (5; 5 / 0)**
immediate repeat / all	10 424	–	**−0.1739 (0; 0 / 5)**
not a repeat / all	1 088 773	–	**+0.0062 (5; 5 / 0)**
1 KC immediate repeat	9 515	–	**−0.1725 (0; 0 / 5)**
1 KC not a repeat	952 201	–	−0.0008 (1; 0 / 4)
multi-KC immediate repeat	909	–	**−0.1877 (0; 0 / 5)**
multi-KC not a repeat	136 572	–	**+0.0258 (5; 5 / 0)**

6.4 Sanity Checks for Leakage

Results of the checks of Section 5.3 (fold 0, seed 42): *Positive control:* flipping the label of a multi-KC question changes the prediction of its later KCs by up to 1.00 for standard DKT (flagged as leaking) and by 0 for DKT-PQC; the next question's prediction still changes in both, so the test is reliable. *Shuffled labels* (Table 5): with labels shuffled by question, only standard DKT learns from the shared label (best validation AUC 0.625); the four variants that satisfy (i) stay at 0.500–0.501. Test Brier separates them less cleanly (0.1631 for DKT and DKT-Fuse, 0.1625–0.1627 for the others), so we rely on validation AUC. With a global shuffle all variants fall to about 0.50, and all five fit 4 training sequences, so low scores under shuffling do not reflect an inability to learn. *Leak-free selection:* it gives the same test result as the default choice (AUC 0.7840 against 0.7841; Tables 2 and 3), so the leakage is not an artifact of checkpoint selection. These controls were run on DKT only; for simpleKT the evidence is the gap between first and later KCs and its removal by the mask (Figure 1).

6.5 First Graded Question of Each Student

The gain is not uniform. For the 3 326 single-KC questions that are the first graded question of a student (0.3% of the test questions), DKT-PQC is 0.0067 *below* DKT: the difference is negative in all 15 runs and its interval excludes 0 in

9 (clear; Table 4). Over all 3 613 first graded questions, including the 287 multi-KC ones (where DKT-PQC is 0.0452 *above* DKT), the difference is +0.0001 and changes sign between runs, so this small group does not contradict the overall gain; we have not tested why standard DKT is better on it. On simpleKT the mask is clearly better on the first graded questions as a whole (+0.0087, 5 of 5 folds; Table 4): the gain on the 287 multi-KC ones (+0.0399) outweighs a small, not distinguishable difference on the single-KC ones (−0.0012).

6.6 simpleKT-PQC versus simpleKT

Three criteria were fixed before the runs: (a) $\Delta\text{AUC} \geq 0.003$ and clear, (b) $\Delta\text{Brier} < 0$ and clear, (c) a gap between later and first KCs within ± 0.03 in all five folds. All three hold. simpleKT-PQC raises AUC from 0.8121 to 0.8164 (+0.0043, from +0.0039 to +0.0050; CI excludes 0 in 5 of 5 folds), and lowers Brier by 0.0020 and ECE from 0.0191 to 0.0081, all clear (Table 2). The gap between later and first KCs, -0.1260 ± 0.0032 for simpleKT, becomes -0.0056 ± 0.0003 (Figure 1): the later KCs gain 0.1196 AUC and the first KC changes by −0.0008. This is the pattern expected if the baseline copies the label of an earlier KC of its own question. Unlike DKT, simpleKT has no shuffle or intervention control, so the evidence agrees with label copying but does not isolate it.

Single-KC questions and immediate repeats. On single-KC questions the mask is slightly worse ($\Delta\text{AUC} = -0.0027$, from −0.0033 to −0.0017; negative in 5 of 5 folds, clear), while on multi-KC questions it is better (+0.0242; Table 4). The loss is concentrated in immediate repeats, i.e. a question attempted twice in a row. On the 9 515 single-KC repeats (0.99% of the group) simpleKT-PQC is 0.1725 AUC below simpleKT in all folds; on the 909 multi-KC repeats it is similar (−0.1877), so it does not depend on the number of KCs. Over all 10 424 repeats (0.95% of the test questions) $\Delta\text{AUC} = -0.1739$ and $\Delta\text{Brier} = +0.0616$ in 5 of 5 folds; ΔECE is +0.0053 with the same sign in all folds but a CI that includes 0 (trend). On the other 99.05% of the questions the gain is larger than overall ($\Delta\text{AUC} = +0.0062$ against +0.0043, 5 of 5 folds). On single-KC questions that are not repeats the difference is −0.0008 (from −0.0014 to +0.0001); its sign changes between folds (not distinguishable by our rule), although the CI lies below 0 in 4 of 5 folds, so a small residual loss cannot be excluded.

A likely cause is our segment convention (Sect. 4.3): consecutive attempts at the same question are merged, so the mask hides the attempt just before the one being predicted, which is legitimate history at test time and highly informative. In fold 0, 98.1% of repeats are correct after a correct attempt, against 33.4% after an incorrect one. The rule does not require this merge, but we have not isolated its effect, and group AUCs are not additive.

7 Discussion and Limitations

Mechanism. On DKT, three independent pieces of evidence point to label copying: the gap between first and later KCs (Fig. 1), the shuffle-by-question control

Table 5. Sanity checks on fold 0 (seed 42), DKT variants. Intervention test: `inside` / `after`. Brier of a constant predictor: about 0.1626.

Check	DKT	DKT-PQC	DKT-ML	DKT-ML-PQC	DKT-Fuse
Intervention, inside / after	1.00 / 0.80 (leaks)	0 / 0.76 (pass)	–	–	–
Overfit 4 sequences, loss after 300 steps	0.0013	0.0013	0.0013	0.0013	0.0014
Shuffle by question: best val. AUC	0.6250	0.5005	0.5004	0.5002	0.5008
Shuffle by question: test Brier	0.1631	0.1625	0.1626	0.1627	0.1631
Global shuffle: best val. AUC	0.5010	0.5007	0.5011	0.5012	0.5009
Global shuffle: test Brier	0.1630	0.1630	0.1627	0.1628	0.1629

(0.625 against 0.500–0.501) and the intervention test (1.00 against 0). On simpleKT only the gap is available, so the evidence is indirect.

Validation in pyKT. The KC-level validation AUC of pyKT exceeds the test AUC by about 0.05 for every model that leaks: +0.052 for DKT and for the pyKT DKT, +0.046 for AKT and +0.049 for simpleKT. For the models that satisfy (ii) and for QIKT the difference is between −0.003 and −0.002. Selecting a model with the pyKT validation would therefore favour a leaking one. This agrees with the inflated validation reported by Badran and Preisach [4].

Size of the effect. The overall effect is small (+0.005 and +0.004 AUC) because only 12.5% of the questions have several KCs.

Limitations. We use one dataset; simpleKT has one seed; the shuffle, intervention and leak-free-selection controls ran on DKT only; no hyperparameter was tuned, and our pyKT baselines are 0.0016–0.0045 AUC below published values. The segment convention of simpleKT-PQC is the likely cause of its loss of 0.17 AUC on immediate repeats and of most of its loss on single-KC questions (Sect. 6.6); we have not isolated its effect, and a small residual loss on single-KC questions that are not repeats (−0.0008) cannot be excluded. DKT-PQC is 0.0067 below DKT on single-KC first graded questions, which we have not explained. We do not rank the three implementations of the rule.

8 Conclusion

In pyKT, multi-KC questions are trained as one token per KC with the question's label but evaluated from the history before the question, so a model can learn to copy the label. Pre-Question Conditioning states what a KC prediction may read: no label of its own question, and the same context as at evaluation. On XES3G5M it raises AUC by +0.0050 on DKT and +0.0043 on simpleKT, lowers ECE, and gives +0.032 and +0.024 on multi-KC questions, while the implementation matters little (at most 0.0006). The evidence for copying is direct on DKT and indirect on simpleKT, where the segment convention also costs AUC on immediate repeats. Future work: exact question boundaries for attention backbones (to test whether they remove this loss), other datasets and models, and question-level validation.

Acknowledgments. This research is funded by ...

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Liu, Z., Liu, Q., Guo, T., Chen, J., Huang, S., Zhao, X., Tang, J., Luo, W., Weng, J.: XES3G5M: A knowledge tracing benchmark dataset with auxiliary information. In: Advances in Neural Information Processing Systems. vol. 36, pp. 32958–32970 (2023)
2. Liu, Z., Liu, Q., Chen, J., Huang, S., Tang, J., Luo, W.: pyKT: A Python library to benchmark deep learning based knowledge tracing models. In: Advances in Neural Information Processing Systems. vol. 35, pp. 18542–18555 (2022)
3. Xiong, X., Zhao, S., Van Inwegen, E.G., Beck, J.E.: Going deeper with deep knowledge tracing. In: Proceedings of the 9th International Conference on Educational Data Mining (EDM). International Educational Data Mining Society (2016)
4. Badran, Y., Preisach, C.: Addressing label leakage in knowledge tracing models. In: du Boulay, B., Mascio, T.D., Tovar, E., Meinel, C. (eds.) Proceedings of the 17th International Conference on Computer Supported Education, CSEDU 2025, Porto, Portugal, April 1-3, 2025, Volume 2. pp. 85–95. SCITEPRESS (2025). https://doi.org/10.5220/0013275200003932
5. Badran, Y., Preisach, C.: Enhancing knowledge tracing through leakage-free and recency-aware embeddings. SN Computer Science **7**(7), 774 (2026)
6. Piech, C., Bassen, J., Huang, J., Ganguli, S., Sahami, M., Guibas, L., Sohl-Dickstein, J.: Deep knowledge tracing. In: Advances in Neural Information Processing Systems. vol. 28 (2015)
7. Liu, Z., Liu, Q., Chen, J., Huang, S., Luo, W.: simplekt: A simple but tough-to-beat baseline for knowledge tracing. In: The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net (2023), https://openreview.net/forum?id=9HiGqC9C-KA
8. Zhang, J., Shi, X., King, I., Yeung, D.Y.: Dynamic key-value memory networks for knowledge tracing. In: Proceedings of the 26th international conference on World Wide Web. pp. 765–774 (2017)
9. Pandey, S., Karypis, G.: A self-attentive model for knowledge tracing. In: Lynch, C.F., Merceron, A., Desmarais, M., Nkambou, R. (eds.) Proceedings of the 12th International Conference on Educational Data Mining (EDM 2019). pp. 384–389. International Educational Data Mining Society (2019)
10. Ghosh, A., Heffernan, N., Lan, A.S.: Context-aware attentive knowledge tracing. In: Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining. pp. 2330–2339 (2020)
11. Chen, J., Liu, Z., Huang, S., Liu, Q., Luo, W.: Improving interpretability of deep sequential knowledge tracing models with question-centric cognitive representations. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 37, pp. 14196–14204 (2023)